Documentatie tehnica

Nedelcu David 322 AA

Aplicația simulează un service pentru electrocasnice, cu gestionarea angajaților, cererilor și tehnicienilor. Structura este orientată pe **OOP** și folosește **moștenire**, **polimorfism** și containere STL.

Clasa	Rol	Relații
`Angajat`	Clasă abstractă pentru orice angajat (Receptioner, Tehnician, Supervizor)	Baza pentru toate clasele de angajați
`Receptioner`	Înregistrează cererile clienților și le stochează într-un vector	Moștenește `Angajat`; comunică cu `Cerere` și `Service`
`Tehnician`	Procesează cererile, calculează salariu și durata totală a lucrărilor	Moștenește `Angajat`; deține vectori cu cereri active și finalizate; interacționează cu `Cerere`
`Supervizor`	Angajat cu rol de supraveghere	Moștenește `Angajat`
`Electrocasnic`	Clasă abstractă pentru aparate (frigider, tv, mașină de spălat)	Bază pentru `Frigider`, `Televizor`, `Masina_de_Spalat`
`Cerere`	Reprezintă o cerere de reparație	Conține pointer către `Tehnician` și `Electrocasnic`
`Service`	Gestionează tot fluxul: angajați, modele, cereri, timp-uri	Comunică cu toate clasele; centralizează logica

Diagrama simplicficata

```
Angajat (abstract)
 ├─ Receptioner
 │   └─ vector<Cerere*>
 ├─ Tehnician
 │   └─ vector<Cerere*> active & finalizate
```

└─ Supervizor

Electrocasnic (abstract)

├─ Frigider

├─ Televizor

└─ Masina_de_Spalat

Cerere

├─ Electrocasnic* aparat

└─ Tehnician* tehnician

Service

├─ vector<Angajat*> Angajati

├─ vector<Tehnician*> Tehnicieni

├─ vector<Electrocasnic*> modele_reparabile

├─ map<pair<string,string>,int> modele_nereparabile

└─ queue<Cerere*> cereri_asteptare

Modul de funcționare

1. **Încarcare angajați și modele reparabile**

- `Service::incarca_angajati()` citește un CSV și creează obiecte `Tehnician`, `Receptioner` sau `Supervizor`.
- `Service::adauga_model()` adaugă modelele reparabile în service.

2. **Înregistrare cereri**
   - `Receptioner::cerere()` adaugă cererile într-un vector intern.
   - `Service::proceseaza_cereri_receptioner()`:
     - Verifică dacă modelul există în `Modele_reparabile`.
     - Dacă există, caută un `Tehnician` disponibil și cu competențele necesare.
     - Dacă nu există tehnician disponibil sau model nereparabil → cererea merge în `Modele_nereparabile`.
3. **Procesare cereri**
   - `Service::timp()` incrementează timpul și apelează `Tehnician::rezolva()`.
   - `Tehnician::rezolva()` reduce durata cererilor active și le mută în `cereri_finalizate` când sunt terminate.
   - `Cerere::timp()` scade timpul rămas pentru cerere.
4. **Salarii și rapoarte**
   - `Tehnician::salariu()` include bază + bonus pentru loialitate, transport și reparații.
   - `Receptioner::salariu()` include bază + bonus loialitate + transport.
   - `Service::top3_salarii()` scrie top 3 angajați în CSV.
   - `Service::durata_max_tehnician()` scrie tehnicianul cu cea mai lungă cerere.
5. **Funcționalități suplimentare**
   - Adăugare și ștergere angajat: `Service::adauga_angajat()`, `Service::stergere_angajat()`.
   - Afișare angajați, tehnicieni, modele reparabile și nereparabile.

Elemente interesante

1. **Forward declaration (`class Tehnician;`)**
   În clasa `Cerere` am folosit *forward declaration* pentru clasa `Tehnician`. Am făcut asta pentru că fiecare cerere trebuie să știe care tehnician o procesează (`Tehnician *tehnician`), dar nu aveam nevoie să includ întreaga definiție a clasei Tehnician în header-ul `Cerere.h`.

Forward declaration îmi permite să declar un pointer către o clasă înainte să știu exact cum arată clasa respectivă. Astfel, evit să includ `Tehnician.h` în `Cerere.h` și reduc dependențele între fișiere, ceea ce grăbește compilarea și previne probleme de tip *circular include*.

2. **Timestamp (`time_t timestamp`)**
   În clasa `Cerere` am adăugat un câmp `timestamp` care stochează momentul creării cererii folosind `time(nullptr)`.
   Acest timestamp îmi permite să știu exact când a fost înregistrată cererea, pentru raportări sau debug.